

1/1/24

MODULE 3

- * Regular Languages
⇒ properties of regular languages
- * pumping lemma for regular language
- * closure properties of regular language
- * Equivalence and Minimization of Finite Automata

Regular languages.

RL are formal languages that regular expressions can describe and can also be recognized by finite automata.

They are used to define sets of strings. Such as sequence of characters or words that follow specific patterns.

They are important in Computer Science and theoretical computer science because they form a foundation for understanding the theory of computation and the design of compilers and other software tools.

Formal Definition

A RL can be defined as the collection of all strings that are recognized by a finite automata

AN FA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$

FAs and RE specify patterns or rules that define a language, such as sequences of characters that must or must not appear in the strings.

The words in a regular language must follow the rules specified by the finite automata or regular expression to be part of the language.

Examples of Regular Languages

- 1) Binary strings that represent even numbers
- 2) Set of strings that contain exactly two a's
- 3) The set of all binary numbers that are divisible by 3
- 4) The set of all strings that contain the substring "01"

Properties of Regular Languages

RL are closed under union, concatenation and Kleene star/closure

This means that if two regular language are combined using one of these operations, the resulting language will also be regular.

⇒ Union: Let L_1 and L_2 be regular language then $L_1 \cup L_2$ is a regular language.

⇒ Concatenation: Let L_1 and L_2 be regular language, then $L_1 \cdot L_2$ is a regular language.

⇒ Kleene star: Let L be a regular language, then L^* is a regular language.

Pumping lemma for RL

The pumping lemma puts forward -d that for a RL L , there exists a constant "pump length" such that any string in the language can be decomposed into three parts and these parts can be repeated any number of times (by "pumping" the middle part) while still being in the language.

This can be stated mathematically as follows.

Let L be a RL and P be the constant "pump length" specified by the pumping lemma. Then for any string w in L such that $|w| \geq P$, it can be decomposed into three parts

$$w = xyz$$

where the following conditions

1. $|xyz| \leq p$

2. $|y| > 0$

3. For any non-negative integer k , xy^kz is in L

⇒ To prove that a language is not Regular using PUMPING LEMMA follow the below steps:

We prove using Contradiction

1) Assume that A is Regular

2) It has to have a pumping length (say p)

3) All strings longer than p can be pumped ($|s| > p$)

4) Now find a string ' s ' in A such that $|s| > p$

5) Divide s into xyz

6) Show that $xy^iz \notin A$ for some i

7) Then consider all ways that s can be divided into xyz

8) Show that none of these can satisfy all the 3 pumping conditions at the same time.

9) s cannot be pumped $\Rightarrow$ CONTRADICTION

Problems

1) Using pumping lemma prove that the language

$A = \{a^n b^n \mid n \geq 0\}$ is Not Regular

proof:

Assume that A is Regular

pumping length = P

$$S = a^P b^P$$

$\overbrace{x} \quad \overbrace{y} \quad \overbrace{z}$

If $P=7 \Rightarrow S = aaaaaaa bbbbbbb$

Case 1: The y is in the 'a' part

$\underbrace{aaaaaaa}_x \underbrace{a}_y \underbrace{bbbbbbb}_z$

Case 2: The y is in the 'b' part

$\underbrace{aaaaaaa}_x \underbrace{b}_y \underbrace{bbbbbb}_z$

Case 3: The y is in the 'a' & 'b' part

$\underbrace{aaaaaaa}_x \underbrace{ab}_y \underbrace{bbbbb}_z$

Case 1: $xy^iz \Rightarrow xy^2z$

$\underbrace{aa}_x \underbrace{aaaa}_{y^2} \underbrace{aaaa}_{y^2} \underbrace{abbbbbbb}_z$

11 a's & 7 b's
no of a's & b's are not equal.

Case 2: $xy^iz \Rightarrow xy^2z$

aaaaaaaabbbbbb b

7 a's 11 b's

No of a's & b's not equal

Case 3: $xy^iz \Rightarrow xy^2z$

aaaaa aabb aabb bbbbbb

9 a's 9 b's

No of a's & b's is equal but the pattern is not followed.

2) Using pumping lemma prove that the language

$A = \{yy \mid y \in \{0,1\}^*\}$ is Not Regular

Proof :-

Assume that A is Regular

pumping length = p

$S = 0^p 1 0^p$

If $p=7$ 0000000100000001

Case 1: 00000000100000001 $xy^iz \Rightarrow xy^2z$

00000000010000001

1st half is not equal to second half of the string the pattern is not followed.

3) using pumping lemma prove that
the language
 $L = \{w w^R \in \{a, b\}^* \mid |w| = 7\}$ is not
regular

Proof: Assume that L is Regular
let the pumping $P = 7$
 $S = w w^R$

suppose $w = abab$
 $w^R = baba$
 $S = abab baba$

Case 1: $\underbrace{abab}_{xy} \underbrace{baba}_z$

Evaluate xy^iz $i=1$ same string
if $i=1$ $abab baba$ $i \geq 0$
if $i=0$ $aabbaba$

if $i=2$ $abbabbaba$

Not a Regular language because the
 $w = abba$ and $w^R = bbaba$
the condition is not satisfied

4) Using pumping lemma prove that
the language
 $L = \{a^n b^{n+1}\}$ is not regular

Proof: Assume that L is regular

Let the pumping length $p = 4$
 $S = a^p b^{p+1}$

Suppose $w = aabbb$
 $S = aabbbbbb$

Case 1: $S = \underbrace{aaa}_x \underbrace{aaa}_y \underbrace{bbb}_z$

Evaluate xy^iz

$i = 2$ $a aaa aaa bbbbbb$

$i = 1$ $a aaa bbbbbb$

$i = 0$ $a bbbbbb$

$$|xy| \leq p$$

$$4 \leq 4 \quad \times$$

No of a's & b's aren't same when

$i = 0$ and the language is also not following the pattern.

Hence the given language is not regular.

5) Using pumping lemma prove that the language

$L = \{a^n b^{2n}\}$ is not regular

Proof: Assume that L is regular

Let the pumping length $p = 4$

Suppose $w = aabbbb$

$S = a^p b^{2p}$

Case 1: $S = \underbrace{aaaa}_x \underbrace{bbbbbb}_y \underbrace{bb}_z$

Evaluate xy^iz

$i = 0$ $a bbbbbbbb$

$i = 1$ $a aaa bbbbbb$

$i = 2$ $a aaa aaa bbbbbb$

$$(xy)^k \neq$$

$$7 < 4 \quad \times$$

No of a's + b's aren't same and the pattern is not followed hence the given language is not regular

c) Using pumping lemma prove that that language $L = \{a^n b b a^n\}$ is not regular

Proof:

Assume that the language is

Regular

Let the pump length = 4

Suppose $S = a b b a$

$S = a^p b b a^p$

$S = aaaa b b aaaa$

Case 1: $S = \underbrace{a a a a}_x \underbrace{b b}_y \underbrace{a a a a}_z$

y in a

Case 2: $S = \underbrace{a a a a}_x \underbrace{b b}_y \underbrace{a a a a}_z$

y in $a b$

Case 3: $S = \underbrace{a a a a}_x \underbrace{b b}_y \underbrace{a a a a}_z$

y in a

Evaluate case 1

$a a a a a a a a b b a a a a$

Evaluate case 3

$a a a a b b a a a a a a a a$

Evaluate case 2

$a a a a b b a b b a a a a a$

The pattern [palindromic] isn't followed in all three cases and the no. of a's & b's aren't of same count.

Hence the given language is not regular.

- 7) Using pumping lemma prove that the language $L = \{a^n b a^n\}$ is not regular

Proof.

Assume that the language is regular

Let the pump length = 4

Suppose $w = a^4 b a^4$

$s = a^p b a^p$

$s = a^{4+4} b a^{4+4}$

Case 1: $s = a^{4+4} b a^{4+4}$

y in a

$s = \underbrace{a^4}_x \underbrace{a^4}_y b \underbrace{a^4}_z \underbrace{a^4}_z$

Evaluate: $a^4 a^4 a^4 b a^4 a^4$

No. of a's & b's isn't same count and the pattern is not followed since the given language is not regular.

Check for all cases if possible.

Closure properties of regular languages

We define closure as a set of things. We describe closure properties as the operations implemented on regular languages, which ensures that a new regular language will be produced.

Properties of regular languages

What are the closure properties of regular languages?

Get Started With Machine Learning

Learn the fundamentals of Machine Learning with this free course. Future-proof your career by adding ML skills to your toolkit — or build foundational ML skills to land a job in AI or Data Science.

[Get Free Course](#)

Overview

educative

Properties of regular languages

All regular languages are closed under the mentioned operations. These operations are as follows:

- Kleen star closure
- Union
- Intersection
- Concatenation
- Complement
- Reversal
- Difference
- Homomorphism
- Inverse homomorphism

Explanation

Let's go over a detailed explanation of the closure properties of regular languages.

Kleen star closure

If a language L_1 is a regular language, then its Kleen closure L_1^* will also be regular.

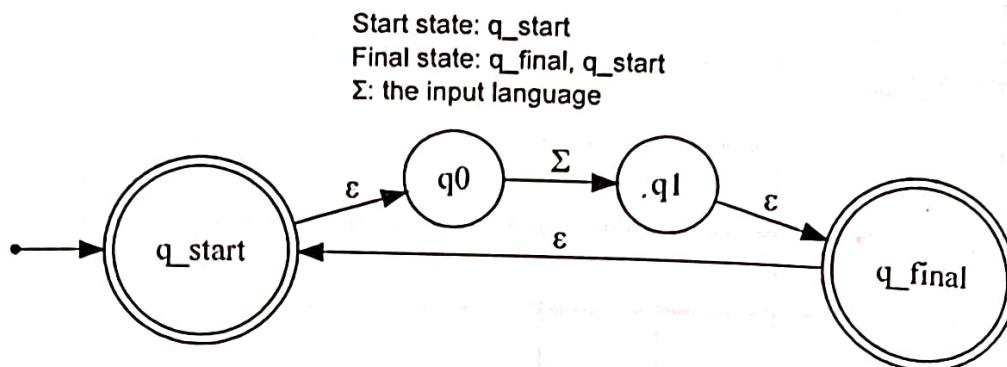
The process for Kleen star closure

To implement Kleen star closure on the language L_1 we follow the steps below:

1. Make a finite automaton for the language.
2. Create a new start state. Join it to the original start state with a null transition.
3. Create a new final state. Join ~~the original final state~~ ^{to new final state} with a null transition. The original final state will now not be a final state anymore.

4. Make a null transition from the new final state to the new start state.

We can illustrate this process in the following way:



The Kleen star closure process

< > ▷ ↶ + []

Union

Let's assume there are two regular languages, L_1 and L_2 . The union of these two regular languages, $L_1 \cup L_2$, is also regular.

We can represent union as $L_1 + L_2$.

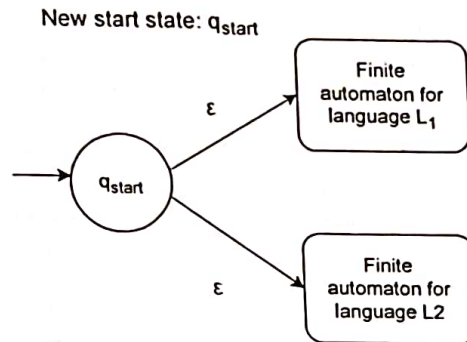
The process for union

Let's assume there are two languages, L_1 and L_2 . We can perform the union of these two languages through the following steps:

1. Make the finite automaton for both L_1 and L_2 separately.

2. Add a new start state. Join the new start state to the automata's original start states by null transitions.

The final states remain the same as for the two automata.



The union process for two languages

2 of

< > ▷ ◀ + []

Intersection

If there are two languages, L_1 and L_2 , the intersection of these regular languages, $L_1 \cap L_2$, is also regular.

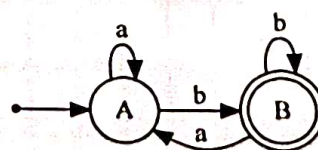
The intersection is checked by De Morgan's law, which states the following: $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$.

The process for the intersection

The process for the intersection is slightly different from other properties. The steps to implement intersection are as follows:

1. Make the automata for both languages.
2. Take the cross-product of all states from both the automata.
3. The final state is the one that has the final state of the first automaton and the second automaton.

We can see this in the illustration below:



$L_1 = \{w \mid w \text{ ends with } b\}$

Concatenation

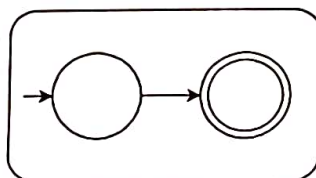
For concatenation, let's assume there are two regular languages, L_1 and L_2 . The concatenation of these two languages, $L_1.L_2$, is also regular.

The process for concatenation

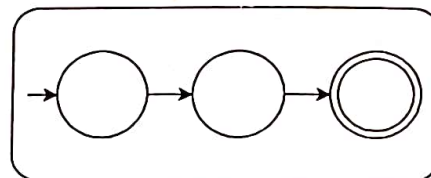
Assume that there are two languages, L_1 and L_2 . We can perform the concatenation of these two languages through the following steps:

1. Make the finite automaton for both L_1 and L_2 separately.
2. Suppose the order of concatenation is $L_1.L_2$. Make a null transition from the final state of L_1 to the starting state of L_2 . Make the final state of L_1 nonfinal.

The final state of L_2 remains. We can see this in the illustration below:



Finite
automaton for
language L_1



Finite
automaton for
language L_2

The concatenation process for two languages

1 of 2

< > ▷ ↶ + {}

Complement

L'_1 represents the complement of the regular language L_1 . Let's assume an alphabet, A , where A^* contains the language, L_1 . The complement is defined as $A^* - L_1$, which is also regular.

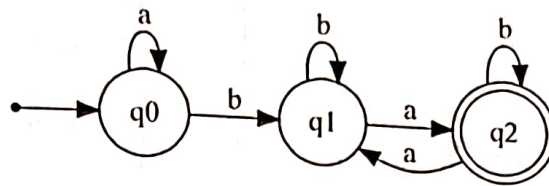
The process for the complement

There is one step for the complement process:

1. Make the accepting states non-accepting and vice versa.

We can see this in the illustration below:

Start state: q_0
Final state: q_2



The process for complementing a language

1 of 2



Reversal

Let's assume a regular language, L_1 . The reverse of L_1 is represented as L_R . The reversed regular language consists of the reverse of all strings present in L_1 .

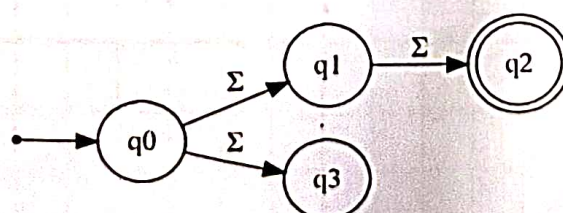
The process for reversal

To reverse a language, we perform the following the steps:

1. Make a definite finite automaton for the regular language.
2. Make the final state the new initial state and the initial state the new final state.
3. Reverse the direction of the transitions.

If there are dangling states, we can remove them from the automaton. We can see this in the illustration below:

Σ : the input language



Difference

Consider the two languages, L_1 and L_2 . The difference between these regular languages is denoted by

$L_1 - L_2$. This means that the strings produced in L_1 are regular and not the ones in L_2 .

This difference is $L_1 \cap \bar{L}_2$ which means the intersection of L_1 and the complement of L_2 .

The process for difference

The process to take the difference between the two languages is as follows:

1. Make the automata for the regular languages, L_1 and L_2 .
2. Take the complement of the second language (the language that is to be subtracted, L_2 in this case).
The steps for complement are explained before.
3. Take the intersection of the language L_1 and $\bar{L}_2$.

Homomorphism

Homomorphism is allowing a string for each input symbol of a language.

For a regular language, L_1 , and the homomorphism, H , the language's alphabet is defined as

$H(L_1) = \{H(s) | \text{where } s \text{ is in } L_1\}$. This is also a regular language.

The process for homomorphism

Homomorphism is closed under regular languages. The algorithm is defined for the regular expression of the language.

- Define homomorphism as the operation on the regular expression of a regular language, L_1 .
- Prove that $L_1(H(R)) = H(L_1(R))$.
- Assume R is such that $L_1 = L_1(R)$. Let $R' = H(R)$, and resultantly $H(L_1) = L_1(R')$.

Example

Suppose that $H(0) = ac$ and $H(1) = bc$.

Let a regular expression of a regular language be $L_1 = 001 + 0^*$.

Then, $H(L_1)$ is the language of $acacbc + ac^*$.

By equating, we see that $H(L_1)$ is also a regular language.

Inverse homomorphism

Consider H as the homomorphism and the regular language, L_1 . The alphabet of L_1 is the output of the language of H . Therefore, $H'(L_1) = \{H(s) | \text{where } s \text{ is in } L_1\}$ is also a regular language.

Equivalence and Minimization of Finite Automata

In automata theory, DFA minimization is the task of transforming a given DFA into an equivalent DFA that has a minimum number of states.

Here, two DFA's are called equivalent if they recognize the same regular language.

Minimization of DFA

DFA minimization stands for converting a given DFA to its equivalent DFA with minimum number of states.

DFA minimization is also called as optimization of DFA and uses partitioning algorithm.

* $\Rightarrow$ Testing whether two descriptions of RE's define the same language.

* $\Rightarrow$ Minimization of DFA's.

$\rightarrow$ Good for implementations of DFA's with less resources (like space, time, IC areas, ...)

1

Testing Equivalence of states

Goal:

We want to understand when two distinct states p and q can be replaced by a single state that behaves like both p and q .

$\Rightarrow$ Two states are said equivalent if for all strings w , $\delta^*(p, w)$ is an accepting state if and only if $\delta^*(q, w)$ is an accepting state.

Note:

It is not necessary to enter the same accepting state for the above definition to be met.

We only require that either both states are accepting or both states are non-accepting.

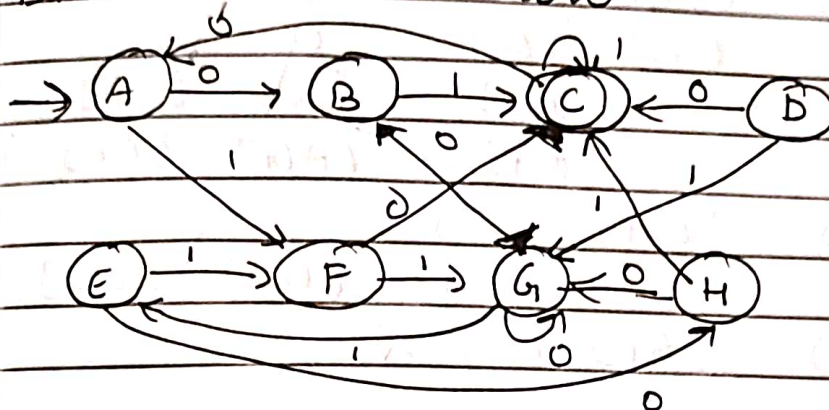
Non-equivalent states are said to be distinguishable.

That is state p is said to be distinguishable from q if there is at least a string w such that one of $\delta^*(p, w)$ and $\delta^*(q, w)$ is accepting and the other is not accepting.

A systematic way to find distinguishable states - use a table filling algorithm.

Example - A

Apply the algorithm to the DFA shown below



Basic: Since C is the only accepting state, we put an x into the pairs of {A, C}, {B, C}, {C, D}, {C, E}, {C, F}, {C, G}, {C, H} with x meaning "distinguishable".

B	x						
C	x	x					
D	x	x	x				
E	✓	x	x	x			
F	x	x	x	x	✓		
G	x	x	x	x	x	x	
H	x	✓	x	x	x	x	x
	A	B	C	D	E	F	G

Induction

A	B	F
B	G	C
C	A	C
D	C	G
E	H	F
F	C	G
G	G	E
H	G	C

		0	1		
B	(B,H)	[GH]	[CC]	✓	
	(B,G)	[GH]	[CE]	X	
	(B,F)	[GC]	[CG]		
		↳ [GC]	[GA]	[EC]	X
		[CG]	[AG]	[CE]	X
	(B,E)	[GH]	[CF]		
		↳ [GH]	[GG]	[CC]	X
	(B,D)	[GC]	[CG]	X	

		0	1		
A	(A,H)	[BG]	[FC]		
		↳ [BG]	[GG]	[CE]	Y
	(A,G)	[BG]	[FE]	X	
	(A,F)	[BG]	[FG]	X	
	(A,E)	[BH]	[FF]	✓	
	(A,D)	[BC]	[FG]	X	
	(A,B)	[BG]	[FC]	X	

D	(D,H)	[CG]	[GC]	X
	(D,G)	[CG]	[GE]	X
	(D,F)	[CC]	[GG]	✓
	(D,E)	[CH]	[GE]	X

E	(E,H)	[HG]	[FC]	X
	(E,G)	[HG]	[FE]	X
	(E,F)	[HC]	[FG]	X

F	(F,H)	[CG]	[GC]	X
	(F,G)	[CG]	[GE]	X

G	(G,H)	[GG]	[GC]	X
---	-------	------	------	---

